

02 - Problema Broadcast

Si consideri un sistema distribuito nel quale una entità vuole condividere con gli altri nodi della rete una informazione che solo lei detiene. Questo problema è detto *broadcasting* e fa parte di una classe generale di problemi detta *information diffusion*.

Risolvere questo problema nel mondo distribuito significa progettare una serie di regole che, quando eseguite dalle entità, portano in tempo finito ad una configurazione nella quale tutte le entità sono a conoscenza dell'informazione. Naturalmente, la soluzione deve funzionare qualunque sia l'entità iniziale che detiene l'informazione.

Come introdotto nella prima lezione, bisogna descrivere formalmente il task specificando:

- L'insieme degli *stati possibili* di ogni nodo;
- l'insieme delle *configurazioni globali iniziali*;
- l'insieme delle *configurazioni globali finali*;
- sotto quali *assunzioni e restrizioni* si analizza il task distribuito.

Problema del Broadcast a singola sorgente

Il task nel problema del **broadcasting** a singola sorgente consiste nel portare il sistema in uno stato in cui tutti i nodi sono informati rispetto ad un messaggio detenuto inizialmente da un singolo nodo del sistema.

Il problema

Dato un grafo $G = (V, E)$, dove V è l'insieme delle entità e G rappresenta la topologia della rete di comunicazione, ed un messaggio $m \in \{0, 1\}^*$ posseduto inizialmente da una singola entità in V , si vuole progettare un protocollo che propaghi m a tutti i nodi.

Si assume che ogni entità $x \in V$ disponga di un registro privato c_x tale per cui

$$c_x = \begin{cases} \text{informed} & \text{se } x \text{ ha ricevuto } m \\ \text{notInformed} & \text{se } x \text{ non ha ricevuto } m \end{cases}$$

Configurazioni del sistema

Le configurazioni del sistema sono tutte le possibili combinazioni degli stati di tutte le entità. L'insieme delle configurazioni iniziali è composto da tutte le configurazioni nelle quali è presente una singola entità informata, corrispondente al nodo che detiene l'informazione da propagare agli altri nodi. Formalmente, l'insieme delle configurazioni iniziali è composto da configurazioni per cui vale il seguente predicato

$$\exists! s \in V : c_s = \text{informed} \wedge \forall v \neq s : c_v = \text{notInformed}$$

Si osserva che l'insieme delle configurazioni iniziali ha cardinalità pari a n .

La configurazione finale invece è unica e corrisponde alla configurazione in cui tutti i nodi

ricevono l'informazione, formalmente è la configurazione in cui

$$\forall v \in V, c_v = \text{informed}$$

Assunzioni sul sistema

Si fanno le seguenti assunzioni o *restrizioni* rispetto al sistema in cui si risolve il problema:

- **Link bidirezionali**, cioè si considera G non diretto;
- **Affidabilità totale**, cioè non ci sono fallimenti;
- **Connettività**, cioè G è connesso. Si osserva che questa restrizione è necessaria: se G è disconnesso, allora alcune entità non possono ricevere l'informazione, dunque il problema del broadcasting sarebbe irrisolvibile;
- **Unique initiator**, cioè solo un'entità può avviare la propagazione del messaggio.

In sintesi, è possibile formalizzare il problema mediante la seguente tripla

$P = \langle P_{init}, P_{final}, R_{flood} \rangle$, dove:

- $P_{init} = \exists! x \in V, c_x = \text{informed} \wedge \forall y \neq x, c_y = \text{notInformed}$
- $P_{final} = \forall x \in V, c_x = \text{informed}$
- $R_{broadcast} = \begin{cases} \text{Total Reliability (TR)} \\ \text{Bidirectional Link (BL)} \\ \text{Connectivity (CN)} \\ \text{Unique initiator (UI)} \end{cases}$

In generale, si indica con R le restrizioni TR, BL, CN. Si ha quindi che $R_{broadcast} = R \cup \{\text{UI}\}$

Note

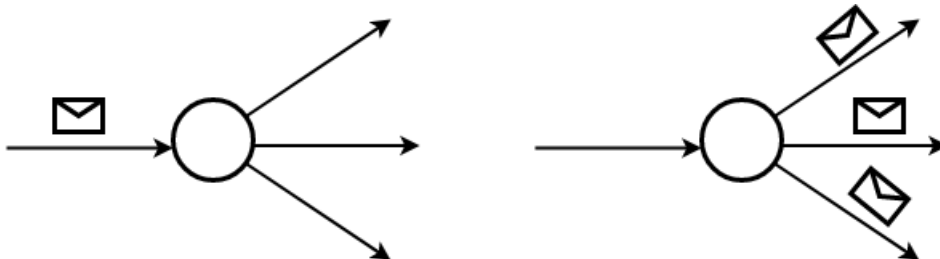
P_{init} e P_{final} sono predicati definiti sull'insieme delle possibili configurazioni del sistema. Si ricorda che la configurazione del sistema in un certo istante è determinata dallo stato assunto da ciascun agente in quel dato istante. In particolar modo,

- P_{init} è un predicato che definisce l'insieme di configurazioni iniziali ammissibili, ossia quelle configurazioni nelle quali il sistema si deve trovare all'inizio dell'esecuzione del protocollo.
- P_{final} è un predicato che definisce l'insieme di configurazioni finali ammissibili, ossia quelle configurazioni nelle quali il sistema si deve trovare al termine dell'esecuzione del protocollo.

Protocollo Flooding

Una prima idea per risolvere il problema è la seguente: se un'entità riceve un messaggio, allora lo inoltra ai suoi vicini. Intuitivamente questa idea farà convergere il sistema verso la

configurazione finale, cioè porterà al completamento del task perché il grafo è connesso e si lavora sotto l'ipotesi dell'affidabilità totale.



Questa prima variante del protocollo Flooding è definita in maniera tale che gli stati assunti dagli agenti durante la sua esecuzione appartengono all'insieme

$\mathcal{S} = \{\text{INITIATOR}, \text{SLEEPING}\}$. Per rappresentare lo stato di un agente si fa riferimento alla funzione

$$\text{status} : V \rightarrow \mathcal{S}$$

Si definisce **INITIATOR** lo stato nel quale si trova l'unico nodo detentore del messaggio da propagare e che deve far iniziare il protocollo. Facendo riferimento alla formalizzazione del problema, il nodo x nello stato **INITIATOR** è l'unico nodo avente inizialmente come valore nel registro c_x impostato a informed ; inoltre si definisce lo stato **SLEEPING** nel quale si trovano tutti gli altri nodi in attesa di ricevere il messaggio. Allora si può descrivere il protocollo eseguito da ogni entità x del sistema nel modo seguente:

```
if state == "INITIATOR": ## stato
    spontaneously: ## evento
        send(message) to N(x) ## azione
elif state == "SLEEPING": ## stato
    receiving(message): ## evento
        send(message) to N(x) ## azione
```

Informalmente, si osserva facilmente che questo protocollo permette al sistema di raggiungere la configurazione finale desiderata, cioè **questo protocollo porta a termine il task richiesto dal problema broadcast in tempo finito**. Ciò nonostante, **il protocollo non termina mai, dato che un qualsiasi nodo nello stato SLEEPING ogni volta che riceve un messaggio lo inoltra, anche dopo averlo ricevuto la prima volta** (si osserva esplicitamente che, nella rete, circola solo un messaggio m).

Inoltre, quando un nodo instrada un messaggio ricevuto a tutto il suo vicinato, rispedisce una copia anche al suo mittente per via dei link bidirezionali. Tale invio è banalmente superfluo.

Per ovviare a questi problemi sono sufficienti le seguenti modifiche:

1. **Se un nodo ha ricevuto almeno una volta il messaggio, dopo averlo inoltrato ai suoi vicini passa in uno stato DONE in cui non effettuerà nessun inoltra di ulteriori messaggi**

ricevuti;

2. Un nodo che riceve un messaggio non lo inoltra al nodo da cui lo ha ricevuto. Per via dei link bidirezionali, un nodo può distinguere i nodi agli estremi dei suoi archi e conosce quali di questi è il mittente del messaggio.
- Il protocollo modificato, dove l'insieme degli stati assunti dai nodi diventa $S = \{\text{INITIATOR}, \text{SLEEPING}, \text{DONE}\}$ è il seguente

```
if state == "INITIATOR":
    spontaneously:
        send(message) to N(x)
        state = "DONE"
elif state == "SLEEPING":
    receiving(message):
        send(message) to N(x) - {source}
        state = "DONE"
elif state == "DONE":
    None
```

Si fa ora un'analisi della correttezza, della terminazione e della *message complexity* del protocollo, dove:

- **Correttezza** si intende verificare che il protocollo porta a termine il task in tempo finito;
- **Terminazione** si intende verificare che il protocollo termina. Si osserva che il primo protocollo progettato porta a termine il task in tempo finito ma non termina mai: i nodi continuano a inviare messaggi sulla rete;
- **Message complexity** si intende analizzare il numero di messaggi scambiati durante l'esecuzione del protocollo.

Correttezza

L'esecuzione del protocollo *flood* sotto le assunzioni

$$R \cup \{\text{Unique initiator}\}$$

dove

$$R = \{\text{Link bidirezionali, Affidabilità totale, } G \text{ connesso}\}$$

è tale per cui esiste un tempo finito $t \in \mathbb{R}^+$ tale che ogni nodo di G ha ricevuto il messaggio m almeno una volta, ovvero

$$\forall v \in V \text{ status}(v) = \text{DONE}$$

Dimostrazione

Sia $\{L_0, L_1, \dots, L_h\}$ una partizione a livelli di V radicata in s , formalmente

$$L_0 = \{s\}$$

$$L_d = \{u \in V - \bigcup_{0 \leq i \leq d-1} L_i : \exists v \in L_{d-1} \text{ t.c. } (v, u) \in E\}$$

per $d = 1, \dots, h$. Dalla definizione di L_d si osserva esplicitamente che

$$\bigcup_{i=0}^h L_i = V \wedge L_i \cap L_j = \emptyset \forall i \neq j$$

L_d è quindi l'insieme dei nodi distanti esattamente d dalla sorgente s . Si dimostra per induzione su d che esiste un tempo t_d tale che tutti i nodi in L_d hanno ricevuto almeno una volta m , ovvero che lo stato dei nodi appartenenti ad L_d al tempo t_d sia **DONE**.

Per $d = 0$ è banale in quanto $L_0 = \{s\}$. Si considera come caso base $d = 1$. Per definizione del protocollo, in un certo istante finito la sorgente invia in maniera spontanea il messaggio a tutti i nodi in $N(s)$ e per definizione di L_d , vale $L_1 \equiv N(s)$. Per l'assunzione dell'**affidabilità totale**, non ci saranno fallimenti durante la trasmissione, quindi, per l'assioma di finite transmission delay, per $d = 1$ esiste un istante $t_1 \in \mathbb{R}^+$ entro il quale tutti i nodi in L_1 verranno informati.

Sia l'ipotesi vera fino ad un certo $d - 1$: si dimostra che l'ipotesi è vera anche per d .

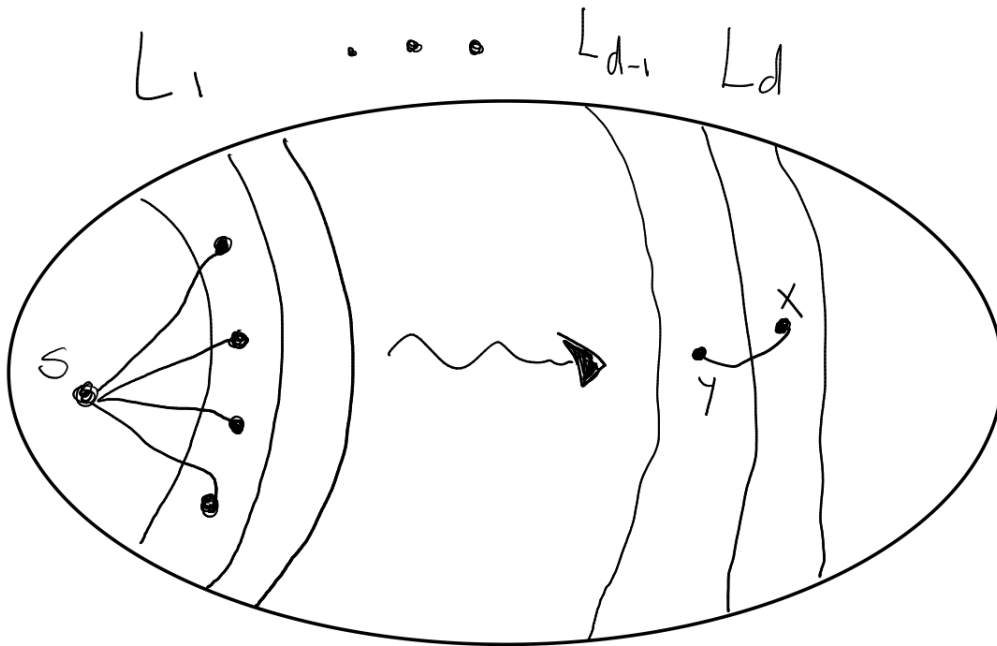
Formalmente, si mostra quindi che

$$\forall x \in L_d, \exists t_d \in \mathbb{R}^+ : \text{status}(x) = \text{DONE}$$

Si consideri un qualsiasi nodo $x \in L_d$ che non si trova già nello stato **DONE**. Per definizione del partizionamento, esiste un nodo $y \in L_{d-1}$ tale che $(y, x) \in E$. Se y è la sorgente s , allora $x \in L_1$ e l'ipotesi vale per il passo base. Se y non è la sorgente, allora certamente y avrà ricevuto il messaggio mentre era nello stato **SLEEPING** per l'ipotesi induttiva. Per definizione del protocollo y avrà inviato il messaggio al suo vicinato $N(y)$, di cui x fa parte. Per l'**affidabilità totale**, il messaggio arriva in tempo finito e non corrotto al nodo x . Infine per l'ipotesi di **connettività** di G , si ha che

$$\forall v \in V, \exists k \in \mathbb{N}, k \leq \text{diam}(G) : v \in L_k$$

Ciò implica che ogni nodo della rete riceve correttamente il messaggio m .



Terminazione

Per definizione, un nodo termina localmente il protocollo quando entra nello stato **DONE**. Si osserva inoltre che un nodo nello stato **DONE** non può transitare in nessun'altro stato. Per la dimostrazione di correttezza del protocollo, che mostra come ciascun agente della rete riceva l'informazione da diffondere, **esiste un tempo $t \in \mathbb{R}^+$ nel quale tutti i nodi si trovano nello stato **DONE****, dunque il protocollo termina globalmente all'istante t .

Note

Le entità **terminano** la propria esecuzione del protocollo in tempi differenti; è quindi possibile che un'entità abbia terminato prima che altre abbiano ancora iniziato. C'è quindi un'importante differenza tra la terminazione locale di un agente, ossia la terminazione dell'esecuzione del protocollo per tale entità, e la terminazione globale, raggiunta quando tutti i partecipanti alla rete terminano.

Inoltre, si fa presente che nel protocollo Flood nessuna entità sa quando l'intero processo è terminato.

Message complexity

In un sistema asincrono, non ha senso parlare di complessità temporale. Dunque si analizza la complessità del protocollo in termini di numero di messaggi trasmessi durante il protocollo.

In prima analisi, si osserva che per via del **ritardo** di trasmissione, su ogni arco possono

essere trasmessi al più due messaggi durante l'intero protocollo. Infatti, un nodo non invierà mai un messaggio attraverso l'arco da cui lo ha ricevuto, ma potrebbe inviare un messaggio su un arco nel quale sta già viaggiando un altro messaggio in entrata per via dei ritardi di trasmissione. Dunque nel caso peggiore vengono inviati due messaggi su ogni arco: il protocollo ha message complexity al più $2m \in O(m)$.

Volendo fare un'analisi più dettagliata, si analizza il numero di messaggi inviati da ogni singolo nodo. Sia $msg(v)$ il numero complessivo di messaggi inviati dal nodo v e sia $M(\text{FLOOD})$ il numero di messaggi trasmessi durante tutto il protocollo. Allora vale

$$\begin{aligned}
 M(\text{FLOOD}) &= \sum_{v \in V} msg(v) \\
 &= msg(s) + \sum_{v \in V - \{s\}} msg(v) \\
 &= |N(s)| + \sum_{v \in V - \{s\}} |N(v)| - 1 \\
 &= |N(s)| + \sum_{v \in V - \{s\}} |N(v)| - \sum_{v \in V - \{s\}} 1 \\
 &= \left(\sum_{v \in V} |N(v)| \right) - (n - 1) \\
 &= \left(\sum_{v \in V} deg(v) \right) - (n - 1) \\
 &= 2m - (n - 1)
 \end{aligned}$$

Dunque il numero di messaggi scambiati è esattamente pari a

$$M(\text{FLOOD}) = 2m - n + 1$$

Complessità temporale: ideal time

In generale, non ha senso calcolare la complessità temporale in un sistema asincrono, a causa dell'impossibilità di poter predire i tempi di trasmissione dei messaggi. Quindi, anche in assenza di guasti, il tempo di esecuzione totale per una computazione è totalmente imprevedibile. Ancor di più, due esecuzioni distinte dello stesso protocollo possono presentare ritardi di trasmissione drasticamente differenti. Dunque per poter analizzare la complessità temporale del protocollo sono necessarie due assunzioni: **clock sincronizzati e bounded delay**.

Si supponga di stare sotto queste due condizioni e si assuma che i messaggi vengano trasmessi con uno stesso ritardo pari ad una unità (unitary communication delay o unitary bounded delay).

Sia $d(x, y)$ la distanza tra il nodo x ed il nodo y in G .

Chiaramente, il messaggio inviato dall'initiator deve raggiungere ogni entità appartenente al sistema, inclusa quella più distante da esso. Sia quindi s la sorgente, l'ideal time complexity è data dall'eccentricità di s , ossia la massima distanza tra s ed un qualsiasi altro vertice in G :

$$T(\text{FLOOD}) = \max_{x \in V} \{d(s, x)\}$$

Quindi, il tempo totale dipende da quale entità svolge il ruolo di sorgente, e non può essere quindi determinata a priori. Si possono però dare dei bound a tale misura osservando che, essendo che ogni entità può svolgere il ruolo di initiator, allora nel worst-case la sorgente s è proprio quella tale per cui

$$\max_{x \in V} \{d(s, x)\} = \text{diam}(G)$$

Allora è possibile dare un upper bound alla complessità temporale: data la sorgente $s \in V$, vale che

$$T(\text{FLOOD}) = \max_{x \in V} \{d(s, x)\} \leq \text{diam}(G) \leq n - 1 \in O(n)$$

Si è quindi data una delimitazione superiore alla complessità temporale del protocollo FLOOD. Si vuole mostrare che tale protocollo sia ottimale in termini di tempo, ossia, che non vi è modo di progettare soluzioni che possano rivelarsi più efficienti di quest'ultima. Per fare ciò, bisogna stabilire un lower bound f tale per cui il costo di ciascun protocollo che risolve correttamente il problema è almeno f . In altri termini, bisogna dare una delimitazione inferiore che non dipenda da un qualsiasi protocollo, ma che dipenda solamente dal problema. Per fare ciò, si osserva che ciascuna entità deve ricevere l'informazione indipendentemente da quanto questa disti dalla sorgente, e che ogni entità può svolgere il ruolo di initiator. Quindi, nel caso peggiore

$$T(\text{Broadcast}/R_{\text{broadcast}}) \geq \max_{x, y \in V} \{d(x, y)\} = \text{diam}(G)$$

Ciò dimostra che FLOOD è un protocollo ottimale per il problema Broadcast, e in particolare modo che:

L'ideal time complexity del broadcasting sotto $R_{\text{broadcast}}$ è $\Theta(\text{diam}(G))$

Note

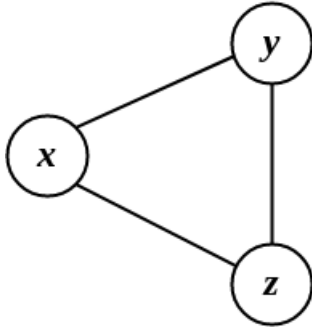
Tempi di trasmissione differenti portano ad esecuzioni diverse dello stesso protocollo, con possibilmente esiti differenti.

Tempo ed eventi

Nel mondo centralizzato, dato uno stesso input ad un algoritmo deterministico, esso non solo genera lo stesso output, ma la sequenza di azioni che svolge per raggiungere tale output è sempre la stessa, ad ogni esecuzione.

Questo dettaglio non vale nel mondo distribuito. Ad esempio nel protocollo flooding, dato che il ritardo dei messaggi varia in modo indeterminato, la sequenza di azioni verso l'unica configurazione finale varia ad ogni esecuzione, anche a fronte di una stessa configurazione iniziale (cioè esecuzioni dallo stesso iniziatore). Naturalmente, per la correttezza del protocollo, una qualsiasi sequenza di azioni anche se diversa converge verso la configurazione finale.

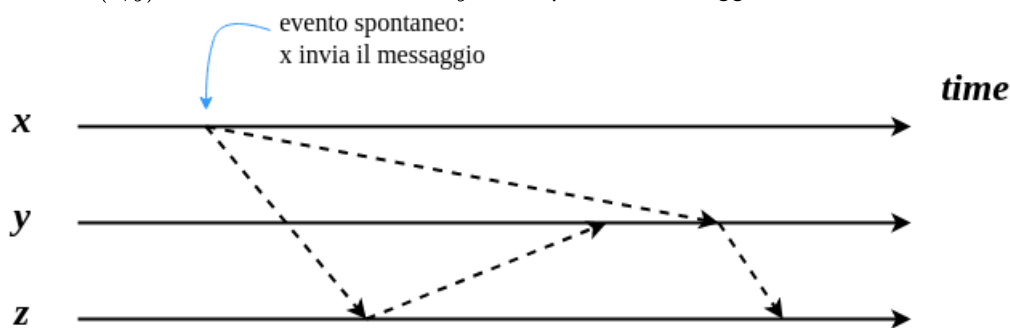
A titolo di esempio, si consideri il seguente grafo dove il nodo x è il nodo iniziatore del protocollo.



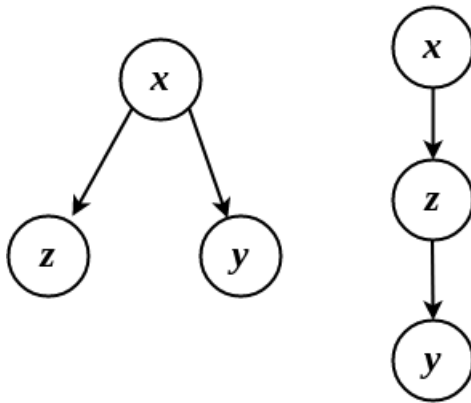
Eseguendo una prima volta il protocollo, potrebbe capitare che i ritardi di trasmissione sugli archi (x, y) e (x, z) siano simili. Perciò una possibile sequenza di azioni potrebbe essere la seguente



Eseguendo il protocollo una seconda volta, potrebbe **capitare** che il ritardo di trasmissione sull'arco (x, y) sia così alto, che alla fine y riceve prima il messaggio tramite z anziché x .



Queste due esecuzioni possono essere rappresentate dai seguenti **grafi di comunicazione**.



Dato che il grafo di comunicazione è sostanzialmente un *albero ricoprente*, ci sono quindi $O(n^n)$ possibili sequenze di trasmissioni (o *traiettorie*) che portano alla configurazione finale.

Lower bound alla message complexity

Il protocollo *flooding* risolve il task del broadcasting con una message complexity pari a $O(m)$. In questa sezione si dimostra che non è possibile progettare un protocollo che risolve il broadcasting con una message complexity $o(m)$.

Definizione (stato locale interno)

Si definisce con $\sigma(x, t)$ lo **stato locale interno** di un nodo x al tempo t .

Per stato interno, si intende qualsiasi informazione contenuta all'interno di x : il contenuto della memoria, dei registri, lo stato ecc.

Fact 1

Se un nodo x si trova in due *ambienti* differenti E_1, E_2 ma $\sigma(x, t)$ è uguale al tempo t sia in E_1 che in E_2 , allora x non è in grado di distinguere i due ambienti ed esegue la stessa azione.

Fact 2

Se due nodi distinti x, y ad un certo istante t si trovano nello stesso stato $\sigma(x, t) = \sigma(y, t)$, allora se vengono sollecitati dallo stesso evento, eseguiranno la stessa azione.

Teorema (Lower Bound Broadcast)

Sotto le ipotesi

$$R \cup \{\text{unique initiator}\}$$

ogni protocollo deterministico per il broadcasting richiede di scambiare **almeno** m messaggi.

Dimostrazione

Si supponga per assurdo che esista un protocollo P con message complexity pari a

$$M(P) = m - 1 < m$$

per ogni grafo $G = (V, E)$ e durante una qualsiasi esecuzione del protocollo.

Allora in G deve esistere un arco $e = (x, y) \in E$ sul quale **non** viene trasmesso alcun messaggio.

Sia $G' = (V', E')$ costruito a partire da G in cui viene rimosso l'arco e , aggiunto un nodo z nello stato **SLEEPING** e aggiunti gli archi (x, z) e (y, z) . Formalmente

$$V' = V \cup \{z\}, \quad E' = (E \setminus \{e\}) \cup \{(x, z), (y, z)\}$$

Dove le porte per x e y sono le stesse in entrambi gli ambienti, formalmente

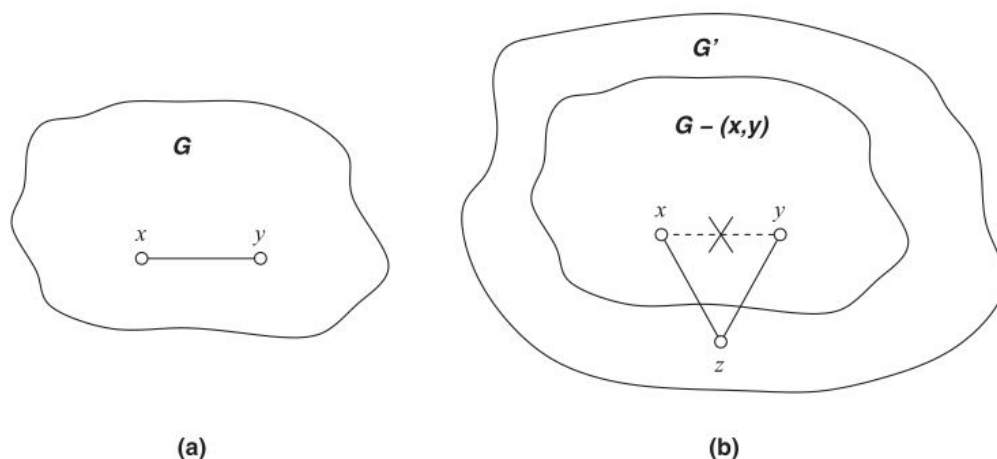
$$\lambda_x(x, y) = \lambda_x(x, z) \wedge \lambda_y(y, x) = \lambda_y(y, z)$$

Si riesegua il protocollo con gli stessi ritardi di trasmissione su G' : essendo che nessun messaggio è stato inviato su (x, y) , ciò risulta possibile.

A livello di stato locale interno, ai due nodi x, y non cambia nulla: le esecuzioni su G e G' sono identiche. Infatti sia x che y in G' vedono un arco uscente laddove in G c'era l'arco (x, y) , ma i nodi non sono in grado di distinguere i vicini agli estremi degli archi (si ricorda che un agente non conosce quali siano i propri vicini all'interno della rete, ma conosce solamente i numeri di porta associati ai link con i quali può comunicare con essi.)

Allora sapendo che da tali porte né x né y trasmettono messaggi in G , ossia nell'esecuzione originale, essendo che il protocollo viene rieseguito con gli stessi ritardi di trasmissione su G' , certamente nessuno dei due trasmetterà sui relativi archi che li collegano a z in G' . Infatti i due ambienti dal punto di vista locale sono **identici** (fact 1), perciò compiono le stesse azioni, ovvero non trasmettere nulla.

Dato che z è collegato solamente a x e y , non può ricevere il messaggio da altri nodi, perciò il nodo z non verrà mai informato, e non passerà mai nello stato **DONE**. Dunque il protocollo P non è corretto, assurdo.



Osservazioni finali

In generale i protocolli di broadcasting sono dispendiosi in termini di messaggi scambiati nel caso in cui vengano eseguiti in una rete *molto densa*, cioè con molti archi. Nel caso limite in cui il grafo è *completo*, il protocollo flooding studiato ha una message complexity pari a $\Theta(n^2)$. Viceversa, nel caso in cui la rete sia un anello o un albero, il protocollo è *ottimale*, cioè la message complexity è pari a $\Theta(n)$.

Allora sembra una buona idea calcolare uno *Spanning tree* della rete nella quale si vuole fare broadcasting e eseguire il protocollo di flooding sullo spanning tree calcolato.

Un problema non banale che occorre però è il calcolo di uno spanning tree in maniera *distribuita*. Inoltre, anche supponendo di poter ottenere uno spanning tree con poche risorse, questa *non è una struttura stabile*. Infatti, in una rete reale capita molto spesso che delle connessioni si interrompano, e se si rimuove un arco da una rete con una struttura ad albero, essa verrà *disconnessa*. In questo caso si dice la rete è **1-tolerant**. In generale, un grafo si dice *k-tolerant* quando anche a fronte della rimozione di *k* archi, esso rimane connesso.

Broadcast su Labeled Hypercube

In questa sezione verrà studiato il problema del broadcasting su *ipercubi* e verrà mostrato come l'aggiunta di conoscenza della topologia della rete da parte dei nodi, combinata alla simmetria della rete stessa, può portare alla creazione protocolli asintoticamente migliori del *Flooding*.

Reti distribuite nel mondo reale

Generalmente in un contesto reale, ad esempio *nel campo delle architetture parallele*, si vogliono costruire reti che abbiano le seguenti proprietà:

1. Un **diametro piccolo**, come $O(1)$ o $\tilde{O}(\log n)$ (poly-log), in modo tale da ridurre i tempi di comunicazione tra i nodi;

2. Il **grafo deve essere sparso**, risparmiando in termini di risorse per la creazione di link. Si osserva che **riducendo il grado massimo dei nodi di un grafo, come conseguenza si ottiene la sua sparsificazione, ma aumenta il suo diametro**.

Un vantaggio importante caratteristico delle reti sparse è la **scalabilità**, ovvero **la capacità di rendere semplice e economica l'aggiunta di nuovi nodi nella rete, mantenendo il diametro piccolo**. Ad esempio, una rete completa non è facilmente scalabile, in quanto ogni volta che viene aggiunto un nodo è necessario creare $n - 1$ nuovi links. Ciò nonostante, **il grafo completo ha diametro pari ad 1**.

Si consideri invece un grafo lineare. L'aggiunta di un nodo comporta l'aggiunta di un solo arco, dunque risulta facilmente scalabile. Nonostante ciò, **il grafo lineare ha come diametro il valore massimo pari a $n - 1$** .

In generale quindi, è utile ricercare una struttura che permetta un buon *trade-off* tra scalabilità e diametro della rete.

Mesh: griglia bidimensionale

Una struttura più equilibrata rispetto alle precedenti è la **rete mesh**, cioè una **griglia bidimensionale**. Infatti in una griglia di n nodi composta da $\sqrt{n}$ righe e colonne, **il grado massimo di ogni nodo è 4, mentre il diametro della rete è pari a $2\sqrt{n}$** . Inoltre ha un buon grado di scalabilità, infatti si possono collegare tra di loro più griglie affiancandole e aggiungendo $\sqrt{n}$ archi. Nonostante il diametro $O(\sqrt{n})$ sia un buon miglioramento rispetto ad n , si vuole ricercare una struttura che garantisca una facile scalabilità e un diametro polilogaritmico nel numero di nodi.

Ipercubo

La struttura che offre un buon compromesso in termini di grado massimo dei nodi, densità, resistenza, diametro e scalabilità è l'**ipercubo**. In particolare, in un **ipercubo di dimensione d** **il numero di nodi è esattamente $n = 2^d$, ed il diametro è pari a $d = \log_2 n$** . Inoltre, ogni nodo ha grado esattamente d , quindi il numero complessivo di archi è $m = \frac{nd}{2} = \frac{n \log_2 n}{2}$.

Broadcast su Labeled Hypercube

Si considera il problema del broadcasting su un ipercubo: oltre alle solite restrizioni R e la restrizione *unique initiator*, in questa variante si assume che **ogni nodo conosce la struttura della rete in cui si trova: un **labeled hypercube** di dimensione d** .

Labeled Hypercube

Un **labeled hypercube** è una particolare rete che ha la struttura di un ipercubo in cui ad ogni nodo e ad ogni arco sono associate delle etichette. In particolare, nodi e archi sono etichettati come segue:

1. Ogni nodo è *univocamente* etichettato con una **stringa** in $\{0, 1\}^d$; Inoltre **ogni nodo ha come vicino i nodi le quali etichette differiscono di un solo bit**, cioè a *distanza di Hamming* pari ad 1;
2. Ogni arco è etichettato con un numero nell'intervallo $[d]$, in modo che tale etichetta rappresenta l'indice del bit per il quale i due nodi estremi differiscono (partendo a contare dal bit meno significativo).

Si ricorda la definizione di *distanza di Hamming*.

Definizione (distanza di Hamming)

Date due stringhe $x, y \in \{0, 1\}^d$, esse si dicono a **distanza di Hamming**

$$d_{\text{Ham}}(x, y) = k$$

se le due stringhe differiscono esattamente di k simboli.

Si definisce formalmente la struttura del Labeled Hypercube

Definizione (Labeled Hypercube)

Un ipercubo etichettato di dimensione d $H_d = (V, E)$ è un grafo il cui insieme dei nodi è l'insieme

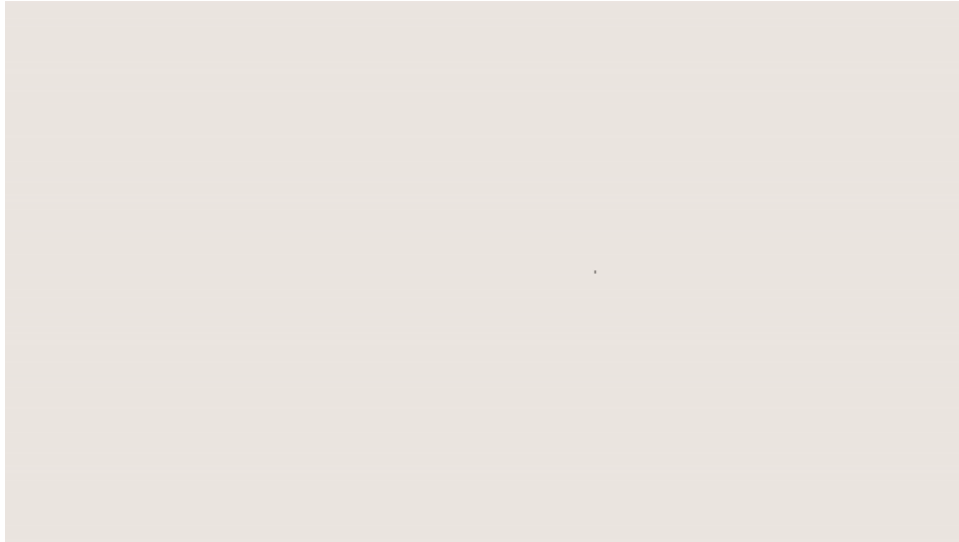
$$V \equiv \{x : x \in \{0, 1\}^d\}$$

mentre l'insieme degli archi è

$$E \equiv \{(x, y) \in V^2 : d_{\text{Ham}}(x, y) = 1\}$$

E' importante far presente che le etichette assegnate ai nodi sono solo per scopo descrittivo, e non sono noti alle entità. Al contrario, le etichette degli archi coincidono con i numeri di porta dei rispettivi nodi, i quali sono noti alle entità per l'assioma di Local Orientation. In particolar modo, si osserva che, $\forall (x, y) \in E, \lambda_x(x, y) = \lambda_y(x, y)$.

Una caratteristica dell'ipercubo etichettato è la sua **scalabilità**. Infatti si può estendere un ipercubo di dimensione d con un altro ipercubo di dimensione d aggiungendo un arco tra i nodi analoghi dei due ipercubi. Di seguito una visualizzazione grafica della generazione di un ipercubo.



Inoltre l'ipercubo è una struttura abbastanza stabile, in quanto per essere disconnessa è necessario rimuovere almeno d archi.

Infine, la caratteristica fondamentale di un ipercubo etichettato di dimensione d è che il diametro è pari a $d = \log_2 n$.

Dimostrazione diametro pari a d

Per dimostrare che il diametro di un ipercubo etichettato H_d è esattamente $d = \log_2 n$, è necessario dimostrare che per ogni coppia di nodi $x, y \in V$ esiste un cammino $P = x \rightsquigarrow y$ tale che la sua lunghezza è minore uguale a d .

Siano $x = \langle x_d, x_{d-1}, \dots, x_j, \dots, x_1 \rangle$ e $y = \langle y_d, y_{d-1}, \dots, y_j, \dots, y_1 \rangle$ due nodi distinti. Allora scorrendo gli indici delle due stringhe a partire dalla posizione meno significativa, si incontrerà un indice j_1 tale che $x_{j_1} \neq y_{j_1}$, essendo x ed y distinti.

Per costruzione dell'ipercubo etichettato, esiste un vicino x' di x tale che la sua etichetta differisce nel j_1 -esimo bit, ossia tale per cui $x'_{j_1} = y_{j_1}$. Si osserva che tale vicino esiste sicuramente, dato che x' e x sono a distanza di Hamming uno. Sia dunque l'arco (x, x') il primo arco del cammino P .

Essendo j_1 il primo indice tale per cui $x_{j_1} \neq y_{j_1}$, ed essendo che x ed x' si distinguono solamente in tale posizione, si ha che per ogni $i = 1, \dots, j_1$ vale che $x'_i = y_i$. Si considerino ora i bit $j_1 + 1, \dots, d$ delle etichette di x' e y . Se tra queste etichette non ci sono bit differenti, allora $x' = y$ e dunque il cammino P è concluso.

Altrimenti, sempre scorrendo verso sinistra, partendo dalla posizione meno significativa, le due stringhe associate ai nodi x' e y , sia j_2 il primo indice incontrato tale che $x'_{j_2} \neq y_{j_2}$ con $d \geq j_2 > j_1$: per l'argomentazione precedente, esiste un vicino x'' di x' tale che $x''_{j_2} = y_{j_2}$.

Sia quindi (x', x'') il secondo arco del cammino P .

Procedendo in questa maniera, prima o poi si dovrà giungere ad un nodo $x^* = y$. Il cammino è costituito da coppie di nodi aventi distanza di Hamming pari ad 1 tra essi, e per ogni arco

da vedere

aggiunto al cammino P ci si avvicina di un salto alla destinazione. Dato che le etichette sono composte da d bit, la distanza tra x e y è al più d , dunque il cammino costruito con tale metodo può avere lunghezza al più d . In particolar modo, dati due nodi x^* ed y^* tali per cui $\forall i = 1, \dots, d, x_i^* \neq y_i^*$, allora il cammino costruito con la procedura appena descritta avrà proprio lunghezza d , e non potendovi essere cammini di lunghezza maggiore di d all'interno del grafo, questo mostra che il diametro è esattamente pari a d .

Protocollo Hyperflood

Tornando al problema del broadcasting, applicando il protocollo Flood su un ipercubo di dimensione d , con $n = 2^d$ nodi, è possibile dimostrare che la message complexity del protocollo è pari a $\mathcal{O}(n \log n)$. Infatti, essendo $m = \frac{n \log_2 n}{2}$:

$$M(\text{FLOOD}/\text{HypCube}) = 2m - n + 1 = n \log_2 n - n + 1 = \mathcal{O}(n \log n)$$

Si progetta ora un protocollo per il broadcast ad hoc per la topologia studiata: **Hyperflood**, in grado di risolvere il task sfruttando fortemente il fatto che i nodi fanno parte di una rete strutturata come un ipercubo etichettato (Non credo proprio). Informalmente, il protocollo funziona nel modo seguente:

- l'unico nodo **INITIATOR** invia il messaggio a tutti i nodi vicini;
 - se un qualsiasi nodo riceve il messaggio da un arco etichettato con il numero l , allora inoltrerà il messaggio a tutti i suoi vicini connessi dagli archi con etichetta $l' < l$.
- Intuitivamente, questo protocollo sfrutta la struttura **gerarchica** dell'ipercubo: il nodo iniziatore invia il messaggio ad un nodo all'interno di ogni sotto-ipercubo di dimensione $d' < d$, e ognuno di questi nodi diventa l'iniziatore del sotto-ipercubo di cui fa parte.

Correttezza e terminazione

Si mostra che il protocollo Hyperflood termina correttamente.

Per tale protocollo, un nodo che riceve il messaggio lo inoltrerà solo ai suoi vicini connessi da archi con etichetta minore rispetto all'etichetta dell'arco da cui è stato ricevuto il messaggio.

Dunque per dimostrare che ogni entità riceve il messaggio inviato dalla sorgente s , si deve dimostrare che, per ogni nodo x , esiste un cammino da s ad x tale che la sequenza degli archi in tale cammino è etichettata in ordine decrescente. A tale scopo, si enuncia e dimostra il seguente lemma

Lemma

In un ipercubo di dimensione d H_d , ogni nodo x è connesso ad ogni altro nodo y tramite un cammino con etichette decrescenti da x a y .

Dimostrazione

Siano $x = \langle x_d, x_{d-1}, \dots, x_1 \rangle$ e $y = \langle y_d, y_{d-1}, \dots, y_1 \rangle$ due nodi distinti. Allora le etichette di x e y differiscono in $t \geq 1$ posizioni.

Siano $j_1, j_2, \dots, j_t$ le posizioni dei bit in ordine decrescente, ossia $j_i > j_{i+1}$. Siano $v_0, v_1, \dots, v_t$ dei nodi dove $v_0 = x$ e dove l'etichetta di v_i differisce dall'etichetta di v_{i+1} solo nella posizione j_{i+1} . Allora esiste un arco etichettato con j_{i+1} che connette v_i e v_{i+1} e chiaramente $v_t = y$.

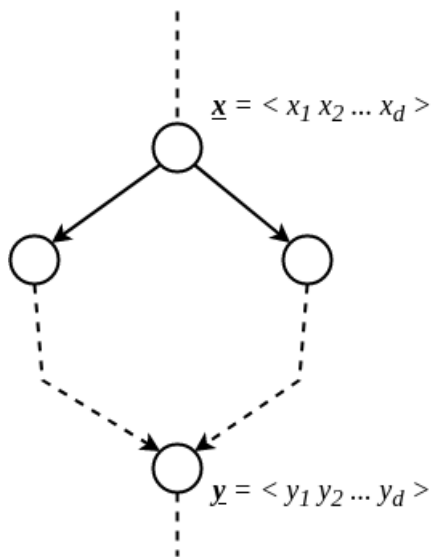
Ma questo implica che $v_0, v_1, \dots, v_t$ è un cammino da x a y e che la sequenza delle etichette degli archi in tale cammino è $j_1, j_2, \dots, j_t$. Ma quest'ultima sequenza è decrescente, dunque il lemma è dimostrato.

Data l'esistenza di questi cammini per ogni coppia s, x con $x \in V - \{s\}$, il grafo delle comunicazioni del protocollo, ossia il grafo residuo ottenuto rimuovendo dall'originale tutti gli archi non coinvolti nello scambio di messaggi, è connesso e contiene tutti i nodi in H_d . Dunque, per quanto appena dimostrato, e per via delle restrizioni del problema, in tempo finito ogni entità riceve il messaggio correttamente; inoltre, per definizione del protocollo, ogni nodo al termine dell'inoltro del messaggio ricevuto entrerà nello stato `DONE`, portando quindi alla terminazione globale.

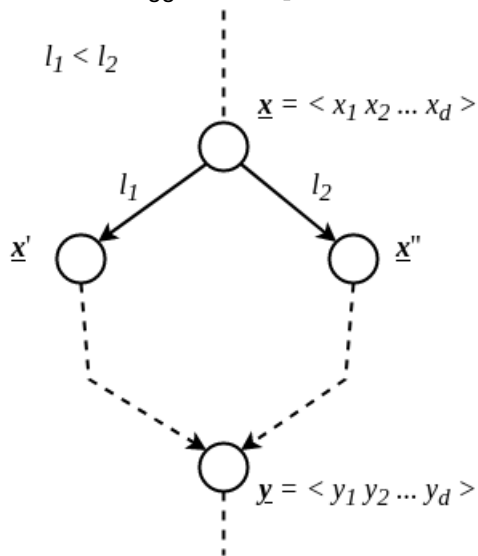
Message complexity

Nel protocollo **Hyperflood** vengono trasmessi esattamente $n - 1$ messaggi, uno per ogni nodo (esclusa la sorgente). Per dimostrare ciò, si dimostra che il grafo delle comunicazioni del protocollo è un **albero** e che quindi ogni nodo non può ricevere più di una volta lo stesso messaggio.

Si supponga per assurdo che esiste un nodo y che riceve due copie del messaggio. Allora nel grafo delle comunicazioni esistono due cammini *disgiunti* che da un nodo x portano al nodo y .



Per costruzione dell'ipercubo etichettato, i due cammini sono tali per cui i loro rispettivi archi iniziali posseggono due etichette distinte l_1 ed l_2 . Si assuma, senza perdita di generalità, che $l_1 < l_2$, e, sempre senza perdita di generalità, si assuma che il cammino di sinistra inizi con l'arco etichettato l_1 , e quello di destra con l'arco etichettato l_2 . Sia x' il nodo raggiunto da l_1 e x'' il nodo raggiunto da l_2 .



Per costruzione vale che:

- x differisce da y almeno per i bits in posizione l_1 e l_2 ;
- $x'_{l_1} = y_{l_1}$ e $x'_{l_2} \neq y_{l_2}$;
- $x''_{l_2} = y_{l_2}$ e $x''_{l_1} \neq y_{l_1}$;

	l_1	l_2	
$\underline{x} =$	$\dots 1 \dots$	$0 \dots$	$>$
$\underline{x}' =$	$\dots 0 \dots$	$0 \dots$	$>$
$\underline{x}'' =$	$\dots 1 \dots$	$1 \dots$	$>$
$\underline{y} =$	$\dots 0 \dots$	$1 \dots$	$>$

Si osserva inoltre che entrambi i cammini procedono per **etichette decrescenti**. Dunque dal cammino di sinistra verranno toccati solo nodi che differiscono da x' per i bits con indice $l_i < l_1$. Questo implica che il cammino di sinistra porta al nodo y senza mai cambiare il bit in **posizione** l_2 , ma questo è assurdo dato che x e y differiscono anche nel bit l_2 .

Ciò dimostra che, dati due nodi della rete, questi non possono essere connessi da più di un cammino nel grafo delle comunicazioni, ossia, tale grafo residuo è un albero. In conclusione, ciascun nodo riceve il messaggio una ed una sola volta, e quindi

$$M(\text{HyperFlood}) = n - 1$$

Complessità temporale

Si analizza ora la complessità temporale in un sistema sincrono nel quale il tempo di trasmissione vale una unità temporale.

Per induzione sulla distanza di Hamming dei nodi dalla sorgente, si dimostra che sotto tali assunzioni la complessità temporale è pari a $d = O(\log n)$.

Dimostrazione

Sia $\{L_0, L_1, \dots, L_h\}$ una partizione a livelli di V radicata in s , formalmente

$$L_0 = \{s\}$$

$$L_k = \{u \in V - \bigcup_{0 \leq i \leq k-1} L_i : \exists v \in L_{k-1} \text{ t.c. } (v, u) \in E\}$$

per $k = 1, \dots, h$. Dalla definizione di L_k si osserva esplicitamente che

$$\bigcup_{i=0}^h L_i = V \wedge L_i \cap L_j = \emptyset \quad \forall i \neq j$$

Si dimostra che per ogni $j \leq d$ e per ogni nodo x a distanza di Hamming j dalla sorgente, il nodo x sarà informato al tempo j .

Per $j = 1$ questo è vero perché per definizione del protocollo la sorgente s invia il messaggio a tutti i suoi vicini, i quali formano l'insieme di tutti i nodi a distanza di Hamming 1 da s .

Si supponga che l'ipotesi sia vera per $j - 1$, ovvero per tutti i nodi in L_{j-1} che sono stati informati al tempo $t = j - 1$. Per ogni nodo $x \in L_j$ esiste sempre un cammino con tutte etichette decrescenti dalla sorgente s ad x . Dato che le etichette sono decrescenti, certamente x è raggiunto da un nodo $y \in L_{j-1}$. Per ipotesi y è stato informato al tempo $j - 1$ e inoltre il messaggio impiega un solo istante per percorrere l'arco (y, x) . Dunque x sarà informato al tempo

$$t = (j - 1) + 1 = j$$

Importante

Il motivo per cui si riesce a migliorare il lower bound $\Omega(m) = \Omega(n \log n)$ sulla message complexity del problema broadcast sotto $R_{broadcast}$ è dato dal fatto che si restringe l'applicabilità del protocollo.

Broadcasting su grafi completi

L'utilizzo del protocollo Flood su un grafo completo G , avente $m = n(n - 1)/2$ e $diam(G) = 1$, richiede $\mathcal{O}(n^2)$ messaggi. Questo è ovviamente non necessario: essendo che, sia $s \in V$ l'initiator, allora $N(s) = V \setminus \{s\}$: è quindi sufficiente che s invii m a tutti i suoi vicini, risolvendo così il task distribuito. Questo protocollo utilizza solo $n - 1$ messaggi e, assumendo sempre bounded delay e synchronized clocks, ideal time pari a $diam(G) = 1$.